

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 1

Реализация REST API на основе boilerplate

Выполнил:
Прель Александр
Группа БР 1.1

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

- 1 Использовать подходящий boilerplate для backend-приложения.
- 2 С учетом Д31 и Д32 реализовать модели, представления и контроллеры.
- 3 Продемонстрировать полностью работающее REST API согласно выбранному варианту приложения.

Ход работы

Для реализации выбран вариант приложения для бронирования столиков в ресторанах. API построено по OpenAPI-спецификации из Д32. Проект размещен в директории БР1.1/Прень Александр/labs/lab1.

Выбранный стек

- Node.js + TypeScript - основной язык и runtime.
- Express - HTTP-сервер и маршрутизация REST API.
- TypeORM - описание сущностей и работа с БД.
- SQLite - локальная учебная база данных для запуска без отдельной настройки СУБД.
- JWT - авторизация защищенных endpoints.
- bcryptjs - хеширование паролей пользователей.

Реализованные модели

- User - пользователь, профиль, email, телефон и password_hash.
- Restaurant - ресторан, адрес, город, ценовая категория и время работы.
- Cuisine - справочник кухонь; связь many-to-many с ресторанами через restaurant_cuisines.
- RestaurantTable - столики ресторана, вместимость и активность.
- Reservation - бронирование столика пользователем.
- Review - отзыв и рейтинг ресторана.
- MenuCategory и MenuItem - категории меню и позиции меню.
- RestaurantPhoto - фотографии ресторанов.

Представления и контроллеры

Представления реализованы как serializer-функции, которые преобразуют TypeORM-сущности во внешний JSON-формат из OpenAPI: User, RestaurantShort, RestaurantDetails, MenuResponse, Review, Reservation и другие. Контроллеры разделены по маршрутам: auth, users, restaurants, cuisines, reservations.

Реализованные endpoints

Группа	Endpoints
Auth	POST /auth/register; POST /auth/login
Users	GET /users/me; PATCH /users/me
Restaurants	GET /restaurants; GET /restaurants/{id}; GET /restaurants/{id}/menu; GET /restaurants/{id}/reviews; POST /restaurants/{id}/reviews; GET /restaurants/{id}/photos
Cuisines	GET /cuisines
Reservations	GET /reservations/me; POST /reservations; GET /reservations/{id}; PATCH /reservations/{id}/cancel

Особенности реализации

- При запуске приложение автоматически создает схему БД через TypeORM synchronize и наполняет ее демонстрационными данными.
- Список ресторанов поддерживает фильтрацию по городу, кухне, ценовой категории, поиск по части названия, сортировку по price_category и rating, а также пагинацию.
- Создание бронирования выполняет автоматический подбор активного столика подходящей вместимости.
- Проверка конфликта бронирований учитывает дату, столик, интервал времени и активные статусы pending/confirmed.
- Защищенные маршруты используют Bearer JWT.
- Ошибки возвращаются в едином JSON-формате с кодами BAD_REQUEST, UNAUTHORIZED, FORBIDDEN, NOT_FOUND, CONFLICT, VALIDATION_ERROR, INTERNAL_SERVER_ERROR.

Проверка работоспособности

Были выполнены команды:

- `npm run typecheck` - TypeScript-проверка прошла успешно.
- `npm run build` - проект успешно собран в `dist`.
- `npm start` - API запущено на локальном порту.

Дополнительно был выполнен HTTP-сценарий на чистой SQLite-базе:

- `GET /health` - 200
- `GET /cuisines` - 200
- `GET /restaurants` с сортировкой `sort_by=rating` и `sort_order=desc` - 200
- `GET /restaurants` с `name=piazza`, `city=Москва`, `price_category=medium`, `sort_by=price_category`, `sort_order=asc` - 200
- `GET /restaurants/1, /menu, /reviews, /photos` - 200
- `POST /auth/register` - 201
- `GET /users/me` - 200
- `PATCH /users/me` - 200
- `POST /restaurants/1/reviews` - 201
- `POST /reservations` - 201
- `GET /reservations/me` и `GET /reservations/{id}` - 200
- `PATCH /reservations/{id}/cancel` - 200
- `GET /unknown-route` - 404 в едином JSON-формате ошибки.

Вывод

В ходе лабораторной работы был реализован backend REST API для сервиса бронирования столиков в ресторанах. Реализация соответствует структуре БД из Д31 и OpenAPI-спецификации из Д32: созданы модели, представления и контроллеры, реализованы регистрация, авторизация, профиль пользователя, поиск ресторанов, меню, фотографии, отзывы и бронирования. Работоспособность подтверждена TypeScript-проверкой, сборкой и HTTP-запросами к основным сценариям API.